



**TECNOLÓGICO  
NACIONAL DE MÉXICO**



**Tecnológico Nacional de México Campus Culiacán  
Ingeniería en sistemas computacionales  
Tópicos de inteligencia artificial**

**José Enrique Félix Esparragoza  
Alfonso Lara Torres**

**Sistema de detección de placas vehiculares  
Dr. Zuriel Dathan Mora Félix**

**8-9 AM**

**30, noviembre, 2025**

## Tabla de contenido

|                                                                   |   |
|-------------------------------------------------------------------|---|
| Sistema de detección de placas vehiculares .....                  | 3 |
| Objetivo General: .....                                           | 3 |
| Objetivos Específicos: .....                                      | 3 |
| Descripción del problema: .....                                   | 3 |
| Justificación:.....                                               | 4 |
| Manual del Usuario Final (Sistema de Detección de Placas) .....   | 4 |
| Propósito del Sistema:.....                                       | 4 |
| Guía de Uso para el Operador:.....                                | 5 |
| Consejos para la Solución de Problemas: .....                     | 5 |
| Documentación Técnica y Manual de Instalación.....                | 6 |
| Especificaciones Técnicas del Sistema .....                       | 6 |
| Arquitectura del Sistema:.....                                    | 6 |
| Esquema de la Base de Datos (Modelos Django): .....               | 7 |
| Lógica de la Aplicación: .....                                    | 7 |
| Configuración de API y Comunicación:.....                         | 8 |
| Video de Funcionamiento: .....                                    | 8 |
| Manual de Instalación .....                                       | 8 |
| Prerrequisitos y Dependencias: .....                              | 8 |
| Instalación del Backend (Servidor API):.....                      | 8 |
| Instalación y Configuración del Cliente (Aplicación Móvil): ..... | 9 |

# Sistema de detección de placas vehiculares

## Objetivo General:

Implementar una solución de software móvil completa para la detección, transcripción (OCR) y validación en tiempo real de matrículas vehiculares. El propósito es migrar la inteligencia artificial (IA) a dispositivos de campo para lograr una alta velocidad de respuesta y automatizar procesos de seguridad y control de acceso, aplicando reglas de negocio específicas como la restricción de acceso tras acumular faltas.

## Objetivos Específicos:

1. Entrenar y optimizar el modelo YOLOv8 para la localización precisa de matrículas.
2. Convertir el modelo a formato TFLite para implementarlo dentro de la aplicación.
3. Utilizar una capa de Reconocimiento Óptico de Caracteres (OCR) que funcione inmediatamente después de la detección para convertir la imagen de la placa en texto (string).
4. Implementar una API RESTful robusta que contenga la lógica para registrar incidencias.
5. Garantizar que el sistema rechace automáticamente el acceso a cualquier vehículo que alcance 3 incidencias.

## Descripción del problema:

La necesidad principal que aborda este proyecto es la falta de sistemas de control de acceso y seguridad vehicular que sean rápidos, móviles y con capacidad de decisión autónoma.

Actualmente, si una organización necesita controlar el acceso o rastrear vehículos sospechosos:

1. Hay lentitud en la identificación: Los sistemas fijos o la identificación manual es lenta y son propensos a errores.
2. No hay control automatizado de faltas: El registro de un vehículo que comete una infracción se hace manualmente o es nulo y la acción administrativa como

el prohibir el acceso depende de una persona revisando una lista, lo cual no es eficaz.

Este proyecto resuelve este problema al ofrecer una solución que opera directamente en el teléfono del usuario, haciendo que sea fácil de utilizar así como al mismo tiempo, ser eficaz.

## Justificación:

La implementación de este sistema se justifica por la necesidad de automatizar la aplicación de normativas y mejorar la convivencia y el orden en espacios privados y públicos, especialmente en la gestión de vehículos.

El proyecto ataca directamente dos problemas sociales y operativos comunes:

1. Invasión de Espacios Reservados y Bloqueos: Es un problema constante que muchas personas no respetan las reglas de estacionamiento, invadiendo áreas de no estacionarse, salidas de emergencia o quitando lugares que son reservados a personas con discapacidades.
2. Necesidad de Control de Faltas Automatizado: Nuestro sistema, al registrar cada falta como una incidencia y aplicar la regla de las 3 incidencias, le da al operador de una herramienta fácil de utilizar. Un vehículo será identificado y se le negará el acceso, haciendo cumplir la regla sin necesidad de que el operador lo recuerde o revise listas.

En resumen, el proyecto utiliza tecnología YOLOv8, TFLite y OCR para resolver problemas cotidianos de orden y seguridad, transformando la detección visual en una acción automatizada y efectiva.

## Manual del Usuario Final (Sistema de Detección de Placas)

### Propósito del Sistema:

El Sistema de Detección de Placas tiene como objetivo registrar y gestionar incidentes relacionados con vehículos y sus usuarios. El sistema permite a los Operadores reportar incidentes en tiempo real y, automáticamente, sanciona a los Usuarios desactivando su estado si acumulan 3 o más incidentes.

## Guía de Uso para el Operador:

El Operador utiliza la aplicación móvil para registrar una Incidencia después de detectar una placa y verificar la información del vehículo y el usuario.

### 1. Identificación del Vehículo

1. Captura de Placa: Utilice la funcionalidad de capturar foto de la aplicación para detectar la matrícula del vehículo.
2. Verificación de Datos: La aplicación buscará el vehículo y su usuario asociado en el sistema de acuerdo al numero de placa.

### 2. Registro de una Incidencia

1. Selección del Operador: Asegúrese de que su ID de Operador se haya ingresado en el campo correspondiente.
2. Selección del Vehículo: Confirme que el vehículo capturado es el correcto.
3. Descripción del Incidente: Ingrese una descripción clara y concisa de la falta cometida.
4. Evidencia Fotográfica: Adjunte la evidencia fotográfica requerida utilizando la camara.
5. Envío: Presione el botón de enviar para registrar la incidencia en el sistema.

Una vez que se registra una incidencia:

- El sistema revisa el total de incidentes acumulados por el Usuario asociado al vehículo.
- Si el número total de incidencias del usuario es mayor o igual a 3, el estado (estado) del Usuario se cambia automáticamente a False (Inactivo), sancionándolo y prohibiéndole el acceso al servicio.

## Consejos para la Solución de Problemas:

### 1. Vehículo no encontrado al capturar la placa

- Causa: La matrícula capturada no se encuentra registrada en la base de datos o fue ingresada incorrectamente.
- Solución: Verifique la matrícula y su correcta lectura. Si la placa es válida, pida al administrador que registre el vehículo en el sistema, asegurándose de asociarlo a un usuario existente.

## 2. Error al registrar una Incidencia

- Causa: El envío de la incidencia carece de campos obligatorios como el ID del operador, la descripción de incidencia, o la evidencia fotografía son inválidos por ejemplo, un numero de matricula u operador que no existe.
- Solución: Revise que todos los campos requeridos en el formulario de la aplicación móvil estén llenos y contengan datos válidos antes de proceder a enviar el registro de la incidencia.

# Documentación Técnica y Manual de Instalación

## Especificaciones Técnicas del Sistema

### Arquitectura del Sistema:

El sistema utiliza una arquitectura Cliente-Servidor (Mobile-Backend), diseñada para la detección y validación de matrículas en tiempo real

- Cliente (Aplicación Móvil): Es una aplicación Android (Kotlin) utilizada por el Operador para la captura de placas y el registro de incidencias. Las tecnologías clave incluyen Kotlin, el Android SDK, y la implementación del modelo de IA en formato TFLite. La comunicación con el servidor se configura a través de constantes, como se define en Constants.kt.

Código fuente: <https://github.com/AlfonsoLT23/Inteligencia-Artificial/tree/main/Unidad%204/PLACAS/app/src/main/java/com/example/placas>

- Backend (API RESTful): Es el servidor que contiene la lógica para registrar incidencias y gestionar los datos del sistema. El backend almacena toda la información de Usuarios, Vehículos, Operadores e Incidencias, utilizando modelos de Django.

Código Fuente: <https://github.com/AlfonsoLT23/Inteligencia-Artificial/tree/main/Unidad%204/backend>

- Inteligencia Artificial (IA): El sistema se basa en un modelo YOLOv8 optimizado para la localización precisa de matrículas. Este modelo se convierte a formato TFLite para su ejecución en la aplicación móvil. Inmediatamente después de la

detección, se utiliza una capa de Reconocimiento Óptico de Caracteres (OCR) para convertir la imagen de la placa en texto.

## Esquema de la Base de Datos (Modelos Django):

El esquema de la base de datos se define en `backend/api/models.py` y consta de las siguientes entidades principales:

- **Usuario:** Representa a la persona asociada a un vehículo. Contiene campos como `nombre_completo`, `tipo_usuario` (con opciones 'alumno', 'empleado', 'otro'), `email`, y el campo booleano `estado` (1 activo / 0 inactivo) crucial para la sanción.
- **Vehiculo:** Identificado por su placa (Primary Key), está asociado a un usuario mediante una clave foránea (ForeignKey) y registra detalles como marca, modelo y color.
- **Operador:** Persona encargada de utilizar la aplicación para registrar incidentes. Contiene su `nombre_completo` y datos de contacto.
- **Incidencia:** Registra una falta. Contiene claves foráneas a `vehiculo` y `operador`, una `descripcion_incidencia`, la `fecha_hora`, y la `evidencia_fotografia`.

## Lógica de la Aplicación:

La lógica de restricción de acceso está automatizada para garantizar que el sistema rechace automáticamente el acceso a cualquier vehículo que alcance 3 incidencias.

- Esta lógica se implementa mediante una señal de Django (`@receiver(post_save)`) que se activa cada vez que se crea un nuevo objeto `Incidencia`.
- El proceso revisa el total de `Incidencia` que tiene el `Usuario` asociado al `Vehiculo`.
- Si el `total_incidencias` del usuario es mayor o igual a 3, el campo `estado` del objeto `Usuario` se cambia automáticamente a `False` (Inactivo), aplicando la sanción.

## Configuración de API y Comunicación:

- URL Base: El cliente se conecta a la API utilizando la constante `BASE_URL`, definida en `src/main/java/com/example/placas/Utils/Constants.kt`. Por defecto, la URL es `http://192.168.1.11:8000/`.
- Seguridad de Red del Cliente: El archivo de configuración `src/main/res/xml/network_security_config.xml` permite explícitamente el tráfico no cifrado (`cleartextTrafficPermitted="true"`) para las IPs locales: 192.168.1.11, 10.0.2.2 (IP del host en emuladores) y localhost, lo cual es necesario para la comunicación durante el desarrollo.

## Video de Funcionamiento:

Demostración del Sistema: <https://github.com/AlfonsoLT23/Inteligencia-Artificial/blob/main/Unidad%204/VideoApp.mp4>

## Manual de Instalación

### Prerrequisitos y Dependencias:

Antes de comenzar, asegúrate de tener instalados Python 3.x y pip para el backend, y Android Studio con el SDK de Java/Kotlin para la aplicación móvil.

### Instalación del Backend (Servidor API):

Estos pasos se realizan dentro de tu IDE de trabajo, utilizando el código del backend.

1. Configurar Dependencias: Instalar los imports necesarios para el proyecto.  
`pip install Django==5.2.8`  
`pip install djangorestframework==3.16.1`
2. Migraciones de Base de Datos:  
Crear los archivos de migración: `python manage.py makemigrations`.  
Aplicar las migraciones a la base de datos: `python manage.py migrate`.
3. Carga de Datos Iniciales: Cargar los datos de prueba o iniciales (Usuarios, Vehículos, Operadores, etc.) en la base de datos: `python manage.py loaddata fixtures/data.json`.

4. Ejecución del Servidor: Iniciar el servidor de desarrollo, especificando la IP de tu máquina para que la aplicación móvil pueda acceder: `python manage.py runserver tuIP:8000`. Debes reemplazar tuIP con la dirección IP local de tu computadora.

## Instalación y Configuración del Cliente (Aplicación Móvil):

Estos pasos se realizan dentro de Android Studio, utilizando el código del cliente.

1. Sincronización: Abrir el proyecto src en Android Studio y realizar la sincronización de Gradle con la opción Sync project
2. Ajuste de Configuración de IP:
  - Archivo Constants.kt: Navega a `src/main/java/com/example/placas/Utils/Constants.kt` y cambia el valor de la constante `BASE_URL` para que coincida con la tuIP que usaste para iniciar el servidor de Django (Ej. `http://tuIP:8000/`).
  - Archivo `network_security_config.xml`: Navega a `src/main/res/xml/network_security_config.xml` y verifica que la IP de tu servidor esté incluida en la configuración de dominio para permitir el tráfico.
3. Compilación y Despliegue:
  - Reconstruir el proyecto con `rebuild project`.
  - Generar el archivo de instalación APK: Construir Apk.
  - Alternativamente, ejecutar la aplicación directamente en un dispositivo o emulador.